Normalization

Database Normalization - 1NF, 2NF, 3NF & BCNF

Normalization is **very commonly asked in DBMS interviews**, especially:

> "Explain normalization with an example."

The best way to answer is not to just define 1NF, 2NF, and 3NF. You should show **how a bad table is progressively decomposed**.

---

1. What is Normalization?

Interview definition

> **Normalization is the process of organizing data in a relational database to reduce redundancy and prevent data anomalies such as insertion, update, and deletion anomalies.**

The main idea is:

```
Bad database
    ↓
Repeated / redundant data
    ↓
Anomalies
    ↓
Normalize
    ↓
Separate related data into tables
    ↓
Connect tables using keys
```

Why do we normalize?

Consider:

Student_ID	Student_Name	Course_ID	Course_Name	Instructor
1	Ali	C101	DB	Ahmed
1	Ali	C102	OS	Sara
2	Sara	C101	DB	Ahmed

Notice:

`Ali` is repeated.

`DB` and `Ahmed` are repeated.

This causes problems.

---

2. The Three Anomalies

Before learning 1NF → 3NF, understand **why normalization exists**.

A. Update Anomaly

Suppose:

Course_ID	Course_Name	Instructor
C101	Database	Ahmed
C101	Database	Ahmed
C101	Database	Ahmed

Ahmed changes his name to "Ahmed Khan".

You have to update multiple rows.

If you update only two:

```
Ahmed
Ahmed Khan
Ahmed Khan
```

Now the database is inconsistent.

That's an **update anomaly**.

B. Insertion Anomaly

Suppose you want to add a new course:

```
C103 | AI | Bilal
```

But no student has enrolled yet.

If your table requires:

```
Student_ID
```

you can't insert the course without inventing student information.

That's an **insertion anomaly**.

C. Deletion Anomaly

Suppose:

Student	Course
Ali	AI
Sara	DB

If Sara drops DB and you delete her row, you might accidentally lose the only information that DB exists.

That's a **deletion anomaly**.

The Goal

Normalization tries to make sure:

```
One fact
   ↓
stored in one appropriate place
```

rather than:

```
Same fact
 ↓
stored in 10 rows
```

---

3. Functional DependencyMUST UNDERSTAND THIS

You cannot properly understand 2NF, 3NF, or BCNF without **functional dependency**.

Suppose:

```
Student_ID → Student_Name
```

This means:

> If I know Student_ID, I can uniquely determine Student_Name.

For example:

```
Student_ID = 101
       ↓
Student_Name = Ali
```

So:

```
101 → Ali
```

Another example

```
Course_ID → Course_Name
```

Knowing the course ID determines the course name.

---

4. What is a Functional Dependency?

Interview statement

> A functional dependency exists when one attribute or set of attributes uniquely determines another attribute.

Written:

```
A → B
```

Read as:

> A determines B.

---

5. Candidate Key

A **candidate key** is a minimal set of attributes that can uniquely identify a row.

Example:

```
Student_ID
```

might uniquely identify a student.

For enrollment:

```
(Student_ID, Course_ID)
```

might uniquely identify an enrollment.

That's a **composite candidate key**.

Now the actual normalization.

1NF - First Normal Form

Rule

A table is in **1NF** if:

1. Each column contains **atomic values**.
2. There are no repeating groups/multi-valued attributes.
3. Each row can be uniquely identified.

The easiest thing to remember:

> **1NF = One cell → One value.**

Or:

> **Atomic values.**

Example - NOT 1NF

Suppose:

Student_ID	Name	Courses
1	Ali	DB, OS, AI
2	Sara	DB, SE

Problem:

`Courses` contains multiple values.

```
Courses
---------
```

```
DB, OS, AI
```

That's not atomic.

Convert to 1NF

Create one row for each course:

Student_ID	Name	Course
1	Ali	DB
1	Ali	OS
1	Ali	AI
2	Sara	DB
2	Sara	SE

Now each cell contains one value.

So:

```
Student_ID | Name | Course
     ↓
Atomic values
```

This is 1NF.

But we have a problem

Look:

```
Ali
Ali
Ali
```

and:

```
Student_ID = 1
Name = Ali
```

is repeated.

Also course information might repeat.

So we haven't solved redundancy completely.

That's where **2NF** comes in.

2NF - Second Normal Form

A table is in **2NF** if:

1. It is already in **1NF**
2. There is **no partial dependency** on a composite candidate key.

This is the important phrase:

> **2NF = No partial dependency.**

What is Partial Dependency?

Suppose our table is:

Student_ID	Course_ID	Student_Name	Course_Name	Grade
1	C101	Ali	DB	A
1	C102	Ali	OS	B
2	C101	Sara	DB	A

Candidate key:

```
(Student_ID, Course_ID)
```

Why?

Because:

```
Student_ID alone
```

doesn't uniquely identify an enrollment.

And:

```
Course_ID alone
```

doesn't uniquely identify an enrollment.

Together:

```
(Student_ID, Course_ID)
```

uniquely identifies it.

Look at dependencies

We have:

```
(Student_ID, Course_ID) → Grade
```

This makes sense.

The student's grade depends on:

```
student + course
```

But:

```
Student_ID → Student_Name
```

Student name depends only on **Student_ID**.

And:

```
Course_ID → Course_Name
```

Course name depends only on **Course_ID**.

But our key is:

```
(Student_ID, Course_ID)
```

So:

```
Student_ID → Student_Name
```

depends on **only part of the composite key**.

That's a:

> Partial dependency.

Why is that bad?

Because we get:

Student_ID	Course_ID	Student_Name
1	C101	Ali
1	C102	Ali
1	C103	Ali

We're storing:

```
Student_ID = 1 → Ali
```

multiple times.

How do we convert to 2NF?

Separate student information:

Students

Student_ID	Student_Name
1	Ali
2	Sara

Courses

Course_ID	Course_Name
C101	DB
C102	OS

Enrollment

Student_ID	Course_ID	Grade
1	C101	A
1	C102	B
2	C101	A

Now:

```
Students
Student_ID → Student_Name

Courses
Course_ID → Course_Name

Enrollment
(Student_ID, Course_ID) → Grade
```

There is no partial dependency.

Therefore:

```
1NF
 ↓
remove partial dependencies
 ↓
2NF
```

Important Interview Point

If a table has a **single-column primary key**, it **cannot have partial dependency** on that key.

Therefore:

> A table in 1NF with a single-attribute candidate key is automatically in 2NF.

This is a very good interview observation.

3NF - Third Normal Form

Now we have to understand **transitive dependency**.

Definition

A table is in 3NF if:

1. It is in 2NF
2. It has **no transitive dependency** of non-key attributes on a key.

Easy memory:

> **3NF = No transitive dependency.**

What is Transitive Dependency?

Suppose:

Student_ID	Student_Name	Dept_ID	Dept_Name
1	Ali	D01	Computer Science
2	Sara	D01	Computer Science
3	John	D02	AI

Primary key:

```
Student_ID
```

Dependencies:

```
Student_ID → Dept_ID
```

and:

```
Dept_ID → Dept_Name
```

Therefore:

```
Student_ID → Dept_ID → Dept_Name
```

So:

```
Student_ID → Dept_Name
```

indirectly.

That's a:

> **Transitive dependency.**

Why is this bad?

`Computer Science` gets repeated:

Student	Dept
Ali	Computer Science
Sara	Computer Science

Suppose department name changes to:

```
Computer Science & Engineering
```

You have to update multiple rows.

Convert to 3NF

Separate departments.

Students

Student_ID	Student_Name	Dept_ID
1	Ali	D01
2	Sara	D01
3	John	D02

Departments

Dept_ID	Dept_Name
D01	Computer Science
D02	AI

Now:

```
Student_ID → Dept_ID
```

and:

```
Dept_ID → Dept_Name
```

are stored in appropriate tables.

The Complete Progression

This is the example I'd recommend memorizing for interviews.

Starting table

```
ENROLLMENT
-----------------------------------------------------
Student_ID
Student_Name
Course_ID
Course_Name
Instructor_ID
Instructor_Name
Grade
```

Suppose key:

```
(Student_ID, Course_ID)
```

Step 1 - 1NF

Ensure:

```
ONE CELL = ONE VALUE
```

No:

```
DB, OS, AI
```

inside one cell.

Step 2 - 2NF

Look for partial dependencies.

```
Student_ID → Student_Name
Course_ID → Course_Name
```

But key is:

```
(Student_ID, Course_ID)
```

Therefore separate:

```
STUDENT
Student_ID → Student_Name

COURSE
Course_ID → Course_Name

ENROLLMENT
(Student_ID, Course_ID) → Grade
```

Step 3 - 3NF

Now look for transitive dependencies.

Suppose:

```
Course_ID → Instructor_ID
Instructor_ID → Instructor_Name
```

Then:

```
Course_ID → Instructor_ID → Instructor_Name
```

So separate:

Course

Course_ID	Instructor_ID
C101	I01
C102	I02

Instructor

Instructor_ID	Instructor_Name
I01	Ahmed
I02	Sara

Now we're at 3NF.

The EASIEST WAY TO REMEMBER

Memorize this:

```
1NF → Atomic
2NF → No Partial Dependency
3NF → No Transitive Dependency
BCNF → Every Determinant is a Candidate Key
```

Or:

> **1 = values, 2 = whole key, 3 = nothing but the key.**

This is a classic mnemonic:

2NF

> Every non-key attribute must depend on the **whole key**, not part of it.

3NF

> Every non-key attribute should depend on **the key, the whole key, and nothing but the key.**

This phrase is excellent for interviews.

BCNF - Boyce-Codd Normal Form

BCNF is a **stronger version of 3NF**.

Definition

> A relation is in BCNF if for every non-trivial functional dependency `X → Y`, X is a superkey.

Easy:

> **Every determinant must be a candidate key/superkey.**

What is a Determinant?

In:

```
A → B
```

`A` is the determinant.

Because A determines B.

So BCNF says:

```
If A → B
then A must be a superkey.
```

Why do we need BCNF if we already have 3NF?

Because:

> Some tables can satisfy 3NF but still violate BCNF.

This is where interviewers sometimes try to catch people.

BCNF Example

Consider:

Student	Course	Instructor
Ali	DB	Ahmed
Sara	DB	Ahmed
Ali	OS	Sara
John	OS	Sara

Assume the business rules are:

1. A student can take many courses.
2. Each course has one instructor.
3. Each instructor teaches only one course.

So:

```
Course → Instructor
```

and:

```
Instructor → Course
```

Because each instructor teaches exactly one course.

Candidate Keys

To identify an enrollment:

```
(Student, Course)
```

can identify a row.

Also:

```
(Student, Instructor)
```

can identify a row because an instructor teaches exactly one course.

Therefore candidate keys include:

```
(Student, Course)
(Student, Instructor)
```

Now look at:

```
Course → Instructor
```

`Course` is NOT a superkey because:

```
Course = DB
```

doesn't identify a particular student.

But:

```
Course → Instructor
```

still exists.

Therefore:

```
determinant = Course
```

is not a superkey.

So it violates BCNF.

How to Fix BCNF?

Decompose:

Course_Instructor

Course	Instructor
DB	Ahmed
OS	Sara

Student_Course

Student	Course
Ali	DB
Sara	DB
Ali	OS
John	OS

Now:

```
Course → Instructor
```

exists in the first table where:

```
Course
```

is the key.

And student-course enrollment is represented separately.

3NF vs BCNF

This is an important interview question.

3NF	BCNF
Less strict	More strict
Allows some dependencies where determinant isn't a superkey under specific conditions	Every determinant must be a superkey
Easier to preserve dependencies	Can require more decomposition
Most practical designs stop around 3NF	Used when stronger normalization is required

The formal 3NF condition for every FD:

```
X → A
```

requires either:

1. `X` is a superkey, **or**
2. `A` is a prime attribute (part of some candidate key).

BCNF removes the second allowance.

So:

```
3NF:
X is superkey
OR
A is prime

BCNF:
X MUST be superkey
```

That's the technically correct distinction.

Complete Comparison

Normal Form	Main Problem Removed	Remember
1NF	Multi-valued/repeating attributes	Atomic values
2NF	Partial dependency	Whole key
3NF	Transitive dependency	Nothing but the key
BCNF	Determinant that isn't a superkey	Every determinant = superkey

One Example to Explain All Four

Suppose interviewer asks:

> "Explain normalization with an example."

You can say:

> "I'll start with a student enrollment table. First, I ensure atomic values to satisfy 1NF. Then, if the table has a composite key such as Student_ID and Course_ID, I remove attributes that depend only on one part of the key to achieve 2NF. Next, I remove transitive dependencies, such as Department_ID determining Department_Name, to achieve 3NF. Finally, BCNF is stricter and requires every determinant in every non-trivial functional dependency to be a superkey."

Then draw:

```
                NORMALIZATION

Unnormalized
      ↓
   1NF
Atomic values
      ↓
   2NF
Remove partial dependencies
      ↓
   3NF
Remove transitive dependencies
      ↓
```

```
  BCNF
Every determinant is a superkey
```

The Best Memory Trick

Remember:

1 → 2 → 3 → BCNF

```
1 = ONE value per cell

2 = depends on the WHOLE key

3 = depends on NOTHING BUT the key

BCNF = determinant must be a KEY
```

Even shorter:

> **1NF = Atomic**
>
> **2NF = Whole key**
>
> **3NF = Nothing but key**
>
> **BCNF = Every determinant is a superkey**

Common Interview Traps

"Does 2NF matter if the primary key is one column?"

No partial dependency is possible with a single-column key, so if the relation is in 1NF, it is automatically in 2NF.

"Is 3NF always BCNF?"

No.

BCNF is stricter.

```
BCNF ⊂ 3NF
```

Every BCNF relation is in 3NF, but not every 3NF relation is in BCNF.

"Is normalization always good?"

Not necessarily.

Highly normalized databases can require more joins.

For read-heavy workloads, you may intentionally **denormalize** certain data.

Good answer:

> "I would normally normalize the transactional schema to maintain consistency, then selectively denormalize or use caching/materialized views when profiling shows a read-performance bottleneck."

"Does normalization improve performance?"

Don't say simply **yes**.

Normalization primarily improves:

- consistency
- data integrity
- maintainability
- reduction of redundancy

It can sometimes hurt read performance because more joins may be required.

Interview-Ready 30-Second Answer

If they suddenly ask **"What is normalization?"**, say:

> **"Normalization is the process of structuring relational tables to reduce data redundancy and avoid insertion, update, and deletion anomalies. In 1NF, values are atomic. In 2NF, there are no partial dependencies on a composite**

> **key. In 3NF, there are no transitive dependencies between non-key attributes. BCNF is stricter than 3NF and requires every determinant in a non-trivial functional dependency to be a superkey. The goal is to store each fact in the appropriate place while maintaining data integrity."**

That is a **very solid interview answer**.

One final thing to drill

For interviews, don't just memorize definitions. Given a table, practice identifying:

```
1. What is the candidate key?
2. What are the functional dependencies?
3. Is it in 1NF?
4. If not → make it 1NF.
5. Is there partial dependency?
6. If yes → decompose to 2NF.
7. Is there transitive dependency?
8. If yes → decompose to 3NF.
9. Does any determinant fail to be a superkey?
10. If yes → BCNF decomposition.
```